in collaboration with

Investigation and Analysis of Computing Data for Data Management

Prince Dangol

Bsc Hons Ethical Hacking & Cyber Security, College of IT & E-Commerce

ST4050CMD Foundations of Computer Science

Submitted to:
Rupak Rujbanshi

March 5th, 2026

ACKNOWLEDGEMENT

I would like to sincerely thank my module supervisor, Sir. Rupak Rajbanshi, for his invaluable guidance, continuous motivation, and professional advice throughout the course of this assignment. His constructive feedback and critical insights helped to shape the direction of this research and significantly improved the quality of my work.

I also extend my gratitude to Softwarica College of IT and E-Commerce and Coventry University for providing the academic resources and institutional support that made this project possible. The experiences gained through this coursework have substantially enriched my knowledge and practical skills in computer science fundamentals.

Finally, I would like to acknowledge my peers and colleagues who supported me, encouraged critical thinking, and motivated the more challenging phases of this assignment.

ABSTRACT

This report presents a comprehensive investigation and analysis of computing data for data management, structured across three tasks aligned with the ST4015CMD Foundations of Computer Science module. Task 1 examines four core encoding formats: Base64, ASCII, Hexadecimal, and URL encoding, analysing their roles in securing data exchange across HTTPS, TLS, and SMTP protocols, with a focus on preventing injection attacks and enabling interoperability in REST APIs and OAuth 2.0 authentication flows. Task 2 investigates the P vs NP computational complexity problem through the lens of a classroom seating arrangement constraint satisfaction problem, demonstrating why verification is $O(n)$ while finding a valid arrangement confronts an $O(n!)$ brute-force search space, and evaluating heuristic strategies that reduce this to $O(n^2)$. Task 3 addresses database normalisation, applying the three normal forms 1NF, 2NF, and 3NF to a college club membership table, constructing an Entity Relationship (ER) diagram, and implementing SQL operations including INSERT, SELECT, and multi-table JOIN queries. Together, these tasks illustrate how encoding formats, computational complexity theory, and relational database design form the theoretical and practical foundations of modern computing data management.

Table of Content

TASK 1: ENCODING FORMATS & SECURE DATA EXCHANGE **6**
1. Introduction **6**
2. Encoding Formats: Strengths, Weaknesses, and Use Cases **6**
2.1 ASCII Encoding 6
2.2 Base64 Encoding 7
2.3 Hexadecimal (Hex) Encoding 7
2.4 URL Encoding (Percent Encoding) 7
3. Role in Securing HTTP Payloads and Preventing Injection Attacks **8**
3.1 Encoding within HTTPS and TLS 8
3.2 Encoding-Based Obfuscation Attacks 8
4. Interoperability with Modern Web Protocols **9**
4.1 REST APIs and OAuth 2.0 9
5. Proposed Enhancements **9**
6. Conclusion **10**
TASK 2: P VS NP & THE CLASSROOM SEATING ARRANGEMENT PROBLEM **10**
1. Introduction **10**
2. The Seating Arrangement Scenario **10**
3. Task 1 P vs NP and the Seating Problem **11**
3.1 What are P and NP? 11
3.2 Why Checking a Plan is Easy and Finding One is Hard 12
3.3 Classification: P or NP? 12
4. Task 2 Brute Force Approach **12**
4.1 How It Works and Why It Fails at Scale 12
5. Task 3 Heuristic (Smart) Approach **13**
5.1 Two Practical Strategies 13
5.2 Time Reduction and Trade-offs 14
6. Conclusion **14**
TASK 3: DATABASE NORMALISATION, ER MODELLING & SQL **15**
1. Introduction **15**
2. Task 1 - Identifying Data Problems **15**
2.1 Three Problems in the Original Table 15
2.2 Anomaly Definitions 16
3. Task 2 Normalising the Data **16**
3.1 First Normal Form (1NF) 16
3.2 Second Normal Form (2NF) 16
3.3 Third Normal Form (3NF) 17
4. Task 3 - Entity Relationship (ER) Diagram **17**
5. Tasks 4 & 5 SQL Operations and JOIN **18**
5.1 Basic SQL Operations (Task 4) 19
5.2 JOIN Query (Task 5) 19
6. Task 6 - Reflection: Normalisation and JOINs **19**
7. Conclusion **20**
REFERENCES **21**

List of Figures

Figure 1: Side-by-Side Comparison of Encoding Formats Strengths, Weaknesses, and Use

Figure 1: Base64 Encoding within a TLS-Secured SMTP Email Transmission Pipeline

Figure 3: Encoding-Based Obfuscation Attack Vectors and Applied Defence Layers..... 8

Figure 4: OAuth 2.0 and JWT Authentication Flow with Base64URL Encoding..... 9

Figure 4: Constraint Graphs Friend Conflicts (left) and City Conflicts (right) Among Figure

Figure 2: Brute Force Factorial Growth Permutation Counts and Feasibility by Class Size... 13

Figure 3: Heuristic Strategies Step-by-Step Flow, Comparison Table, and Example Seating

Figure 1: Data Problems in the Original Unnormalised Club Membership Table..... 15

Figure 2: Normalisation Steps Unnormalised Table through 1NF, 2NF, and 3NF with Formal

Figure 3: ER Diagram - College Club Membership System (Entities, Attributes, PK/FK,

Figure 4: SQL INSERT, SELECT, and JOIN Queries for the Normalised Club Membership

TASK 1: ENCODING FORMATS & SECURE DATA EXCHANGE

1. Introduction

Encoding formats are foundational tools in modern networked communication. They transform raw data binary files, credentials, and structured payloads into representations that can safely traverse the text-based protocols underpinning the internet. This report investigates four core encoding formats: Base64, ASCII, Hexadecimal (Hex), and URL encoding, analysing their role within secure web protocols including HTTPS, TLS, and SMTP.

The report evaluates each format's strengths and weaknesses, examines its role in preventing injection attacks, assesses interoperability with REST APIs and OAuth 2.0, and proposes enhanced encoding strategies. Real-world scenarios ground the analysis throughout: email attachment transmission, web authentication flows, and REST API file transfers.

2. Encoding Formats: Strengths, Weaknesses, and Use Cases

Figure 1 provides a comparative overview of all four encoding formats discussed in this section.

Figure 1: Side-by-Side Comparison of Encoding Formats Strengths, Weaknesses, and Use Cases

2.1 ASCII Encoding

ASCII maps 128 characters to 7-bit integers (0–127). Adopted across virtually every operating system, programming language, and communication protocol, it provides unmatched cross-platform interoperability. SMTP, HTTP headers, and most configuration formats were designed around ASCII-safe text.

Strengths: Universally supported with zero processing overhead; foundational standard enabling seamless interoperability between legacy and modern systems; basis for all subsequent encoding standards.

Weaknesses: Limited to 128 characters; cannot represent binary data, images, or non-Latin scripts.

2.2 Base64 Encoding

Base64 converts arbitrary binary data into 64 printable ASCII characters, grouping every three input bytes into four 6-bit groups. It is the industry standard for SSL/TLS certificate content and is mandated by MIME (RFC 2045) for encoding binary email attachments; without it, binary content is corrupted passing through SMTP gateways that strip 8-bit data (Josefsson, 2006).

```
HTTP Basic Auth:  Authorization: Basic dXNlcjpwYXNzd29yZA==
```

Strengths: Guarantees binary data survives text-only channels; universally supported across HTTP clients, mail servers, and cryptographic systems; standard encoding for PEM certificates, MIME attachments, and JWT tokens; enables cross-platform interoperability without platform-specific binary handling.

Weaknesses: Increases payload size by ~33%; provides no confidentiality. Base64 is trivially reversible and must never be confused with encryption.

2.3 Hexadecimal (Hex) Encoding

Hex encoding represents each byte as two hexadecimal digits (0–9, A–F). SHA-256 hashes, TLS session identifiers, MAC addresses, and HTTP cookie values are conventionally expressed in hex, making it the standard representation for cryptographic outputs and network protocol debugging.

```
Example:  ASCII '/'  =  Decimal 47  =  Hex '2F'
```

Strengths: Human-readable byte-level representation; universally used for hash outputs and cryptographic identifiers.

Weaknesses: Doubles the size of the original binary. Attackers exploit hex encoding to obfuscate XSS payloads encoding '<' as < to bypass string-matching filters (OWASP, 2023).

2.4 URL Encoding (Percent Encoding)

URL encoding (RFC 3986) replaces structurally significant URI characters with '%' followed by their two-digit hexadecimal value. Browsers apply this automatically to form submissions and query strings, preventing user-supplied data from being misinterpreted as URI elements.

```
Example:  'search=Fish & Chips'  →  'search=Fish+%26+Chips'
```

Strengths: Maintains URI structural integrity; universally implemented in all browsers and HTTP clients; partial defence against certain injection vectors.

Weaknesses: Vulnerable to double-encoding attacks ('%25' for '%'), enabling WAF bypass. The IIS Unicode bug (CVE-2001-0333) exploited double-encoded path traversal for remote code execution (HackerOne, 2024).

3. Role in Securing HTTP Payloads and Preventing Injection Attacks

3.1 Encoding within HTTPS and TLS

HTTPS layers HTTP over TLS (v1.3, RFC 8446), providing confidentiality, integrity, and authentication. Encoding and TLS are architecturally complementary: encoding ensures format compatibility; TLS ensures confidentiality for the encoded data in transit. Figure 1 illustrates Base64 encoding within a TLS-secured SMTP flow.

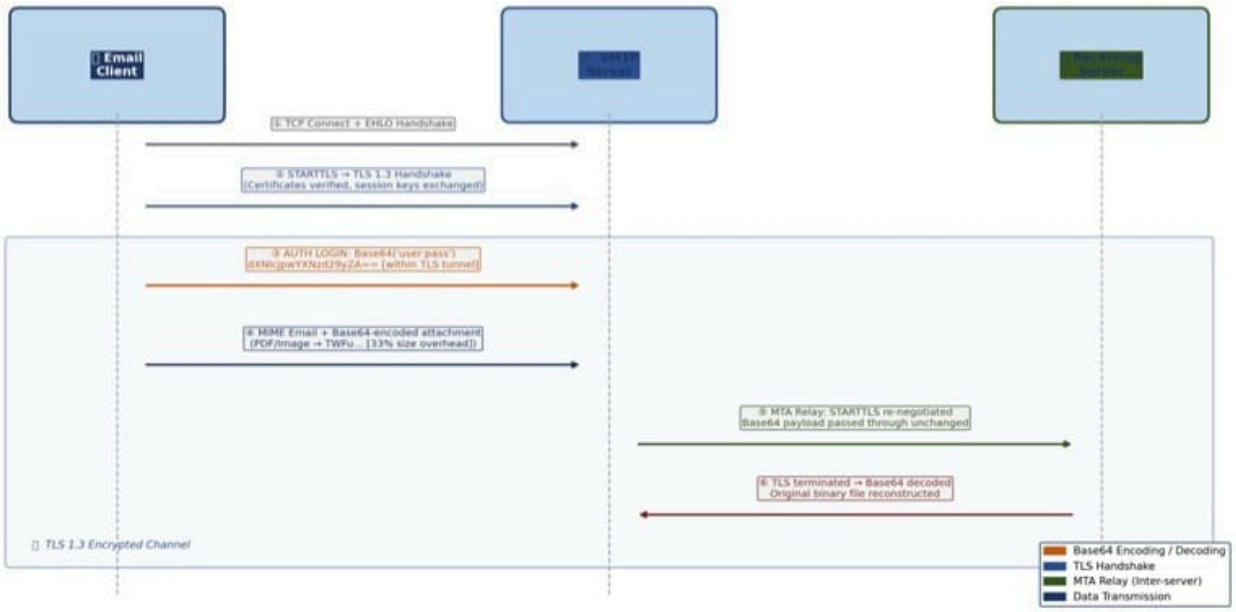

Figure 1: Base64 Encoding within a TLS-Secured SMTP Email Transmission Pipeline (STARTTLS Flow)

3.2 Encoding-Based Obfuscation Attacks

Encoding formats are also weaponised to obfuscate malicious payloads. Figure 3 maps the principal attack vectors and defence layers identified by OWASP (2023):

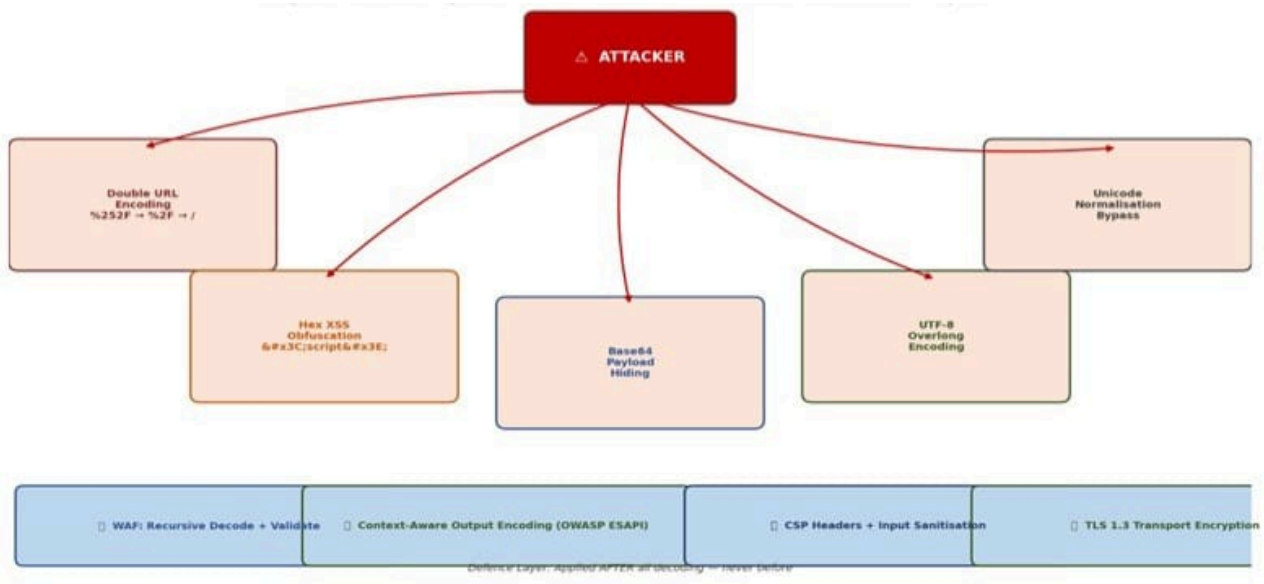

Figure 3: Encoding-Based Obfuscation Attack Vectors and Applied Defence Layers

- Double URL Encoding: '%' encoded as '%25' produces '%252F' for '/'. A WAF decoding once sees '%2F'; the back-end decodes fully to '/', enabling path traversal.
- Hex-Encoded XSS: '<' and '>' expressed as < and > evade string-matching filters but execute as HTML in the browser.
- UTF-8 Overlong Encoding: '/' (0x2F) encoded as C0 AF exploits servers that evaluate multibyte forms before enforcing access controls.
- Base64-Obfuscated Malware: Command-and-control payloads encoded as Base64 blend into legitimate HTTPS or email traffic.

4. Interoperability with Modern Web Protocols

4.1 REST APIs and OAuth 2.0

JSON cannot natively carry binary data. Base64 bridges this gap. The pattern of Base64 for binary compatibility, JSON for structure, and TLS for confidentiality is the de facto standard for modern API design. OAuth 2.0 uses JSON Web Tokens (JWTs) with three Base64URL-encoded sections (header, payload, signature). Base64URL (RFC 4648) replaces '+' with '-' and '/' with '_', making tokens safe for HTTP headers and URL parameters without additional percent-encoding (Josefsson, 2006). Figure 4 illustrates the complete OAuth 2.0 flow.

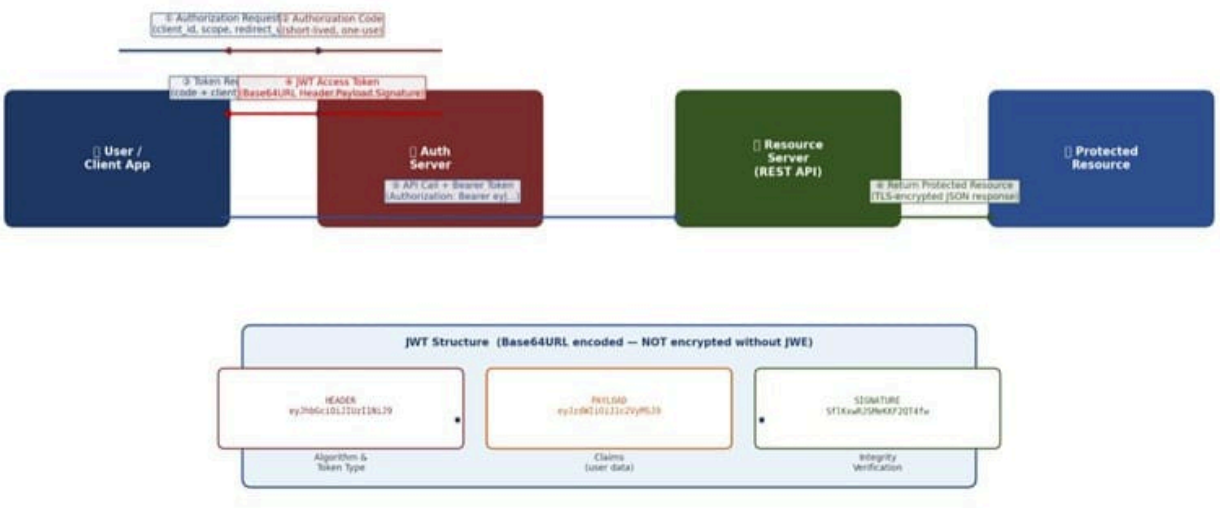

Figure 4: OAuth 2.0 and JWT Authentication Flow with Base64URL Encoding

As shown in Figure 4, JWT payloads are Base64URL-encoded but not encrypted. TLS is mandatory for all JWT-bearing traffic. In 2024, Microsoft retired SMTP Basic Authentication (Base64-encoded credentials) in favour of OAuth 2.0 via XOAUTH2, confirming that encoding-only credential transmission is insufficient for modern security.

5. Proposed Enhancements

- Adopt Base64URL universally (RFC 4648 §5): Eliminates additional percent-encoding in URLs and headers, reducing payload size without altering security properties.

- Enforce context-aware encoding (OWASP ESAPI): Different schemes per output context, HTML entity encoding in HTML, Unicode escaping in JavaScript, percent encoding in URLs, and closing layer-mismatch vulnerabilities.
- Pair encoding with authenticated encryption (AES-GCM + Base64): Encrypt sensitive payloads with AES-GCM before Base64 encoding, as formalised in JWE (RFC 7516).
- Replace Basic Auth with OAuth 2.0: Eliminate Base64-encoded credential transmission across SMTP, HTTP, and IMAP in favour of short-lived, cryptographically signed tokens over TLS 1.2+.

6. Conclusion

Encoding formats, such as Base64, ASCII, Hex, and URL encoding, are indispensable components of modern secure data exchange. Their greatest collective strength is universal interoperability: they provide a common language through which binary data, credentials, and structured payloads traverse diverse systems without corruption. This is evident across all three real-world scenarios examined: Base64 enables binary attachments to survive multi-hop SMTP relay chains (Figure 1); URL encoding ensures query parameter integrity in REST API and OAuth 2.0 flows (Figure 4); and together these formats underpin the certificate and token infrastructure securing HTTPS web authentication at scale. The essential distinction encoding provides format compatibility, and confidentiality must guide every implementation decision.

TASK 2: P VS NP & THE CLASSROOM SEATING ARRANGEMENT PROBLEM

1. Introduction

A teacher must seat n students in a single row before an examination, subject to two constraints: friends must not sit adjacent to each other, and students from the same city must not sit next to each other. Despite sounding straightforward, this problem illustrates one of the deepest unsolved questions in computer science, the P vs NP problem, formalised by Stephen Cook in 1971 (Sipser, 2012). This report analyses the seating arrangement from three angles: complexity theory, brute force enumeration, and heuristic strategies, supported by Figures 1 – 4.

2. The Seating Arrangement Scenario

Formal Problem Statement

Assign each of n students a unique seat in a single row such that: (1) No two friends occupy adjacent seats. (2) No two students from the same city occupy adjacent seats. Decision question: Does at least one valid arrangement exist?

This is a constraint satisfaction problem (CSP). Figure 4 visualises the constraint relationships between students, making the structural complexity immediately apparent.

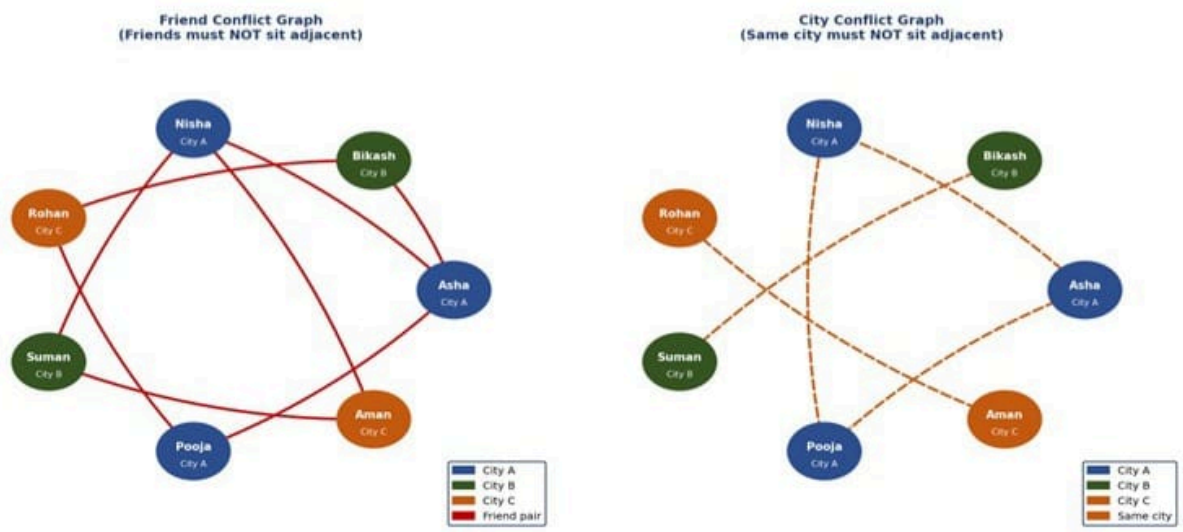

Figure 4: Constraint Graphs Friend Conflicts (left) and City Conflicts (right) Among Students

As shown in Figure 4, each edge represents a pair who must not sit adjacent. The dense, overlapping constraints grow with n, foreshadowing the search difficulty explored below.

3. Task 1 - P vs NP and the Seating Problem

3.1 What are P and NP?

Class P Polynomial Time

Problems solvable in polynomial time (O(n), O(n²), etc.). Examples: sorting, binary search, shortest-path algorithms.

Class NP Non-deterministic Polynomial Time

Problems whose proposed solutions can be VERIFIED in polynomial time, even if no efficient finding algorithm exists. Examples: Boolean SAT, Graph Colouring, Travelling Salesman, and the Seating Arrangement Problem.

All P problems are within NP. Whether NP contains problems genuinely outside P is unproven. Figure 1 illustrates this relationship and positions the seating problem within it.

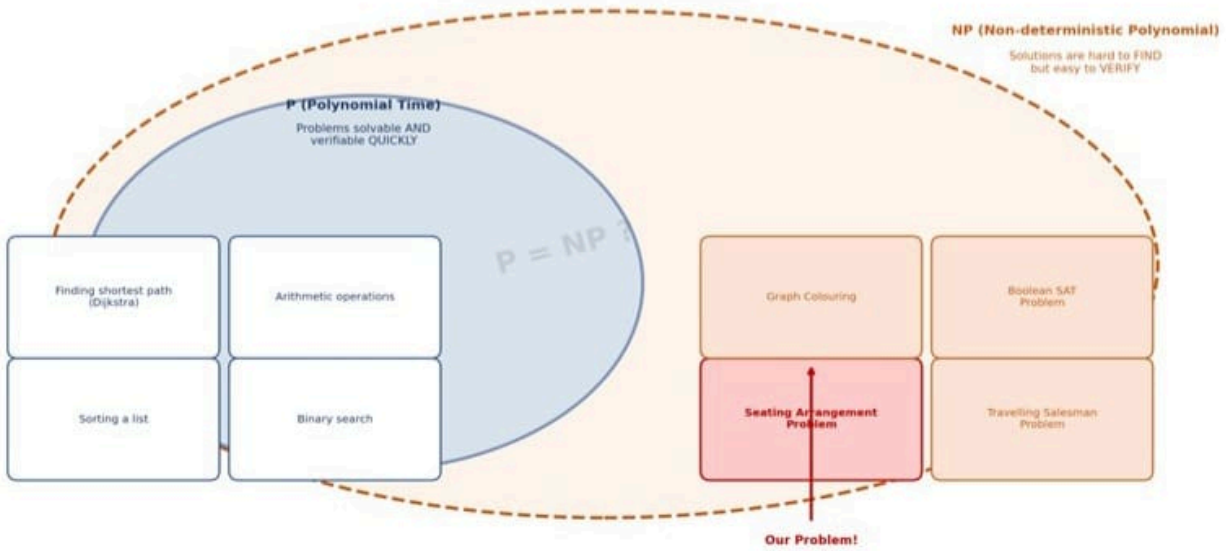

Figure 1: P vs NP Complexity Classes. The Seating Arrangement Problem Sits Within NP

3.2 Why Checking a Plan is Easy and Finding One is Hard

Verifying a proposed arrangement requires a single pass through (n − 1) adjacent pairs, each involving two O(1) lookups. Total verification time is O(n), firmly in Class P (Cormen et al., 2022).

```
Verify:  for i = 0 to n-2:
           if friends(seat[i], seat[i+1]) or sameCity(seat[i], seat[i+1]):
INVALID
         return VALID
```

Finding a valid arrangement is different. With n students, there are n! permutations. The constraints interact, placing a student in seat 1 restricts seat 2, which restricts seat 3, potentially forcing backtracking through exponentially many partial arrangements. The problem is equivalent to finding a Hamiltonian path in the complement of the constraint graph (Figure 4), a known NP-complete problem (Garey and Johnson, 1979).

3.3 Classification: P or NP?

The seating arrangement problem is NP, confirmed by three observations:

1. Verification is O(n), placing it within NP by definition.
2. No polynomial-time finding algorithm is known for all inputs.
3. The problem reduces to Graph Colouring, a known NP-complete problem (Garey and Johnson, 1979).

4. Task 2 Brute Force Approach

4.1 How It Works and Why It Fails at Scale

Brute force generates every permutation of n students, checks each against both constraints, and returns the first valid arrangement. Figure 2 shows both the raw permutation counts and the practical feasibility of increasing class sizes.

Students (n)	Arrangements (n!)	Feasibility
1	1	Instant
2	2	Instant
3	6	Instant
4	24	Instant
5	120	Instant
6	720	Instant
7	5,040	Fast
8	40,320	Fast
9	362,880	Fast
10	3,628,800	Slow
11	39,916,800	Slow
12	479,001,600	Slow

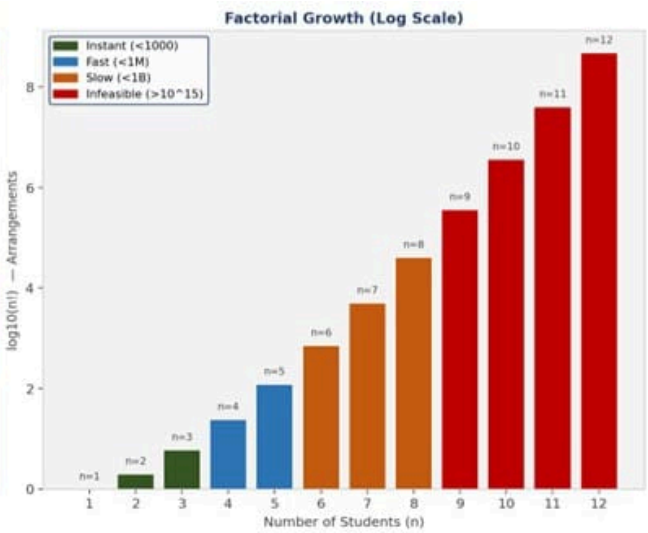

Figure 2: Brute Force Factorial Growth Permutation Counts and Feasibility by Class Size

Students (n)	Arrangements (n!)	Estimated Time	Feasibility
5	120	< 1 microsecond	Instant
8	40,320	< 1 millisecond	Fast
10	3,628,800	~4 milliseconds	Manageable
12	479,001,600	~0.5 seconds	Slow
15	1.3 trillion	~22 minutes	Very Slow
20	2.4 quintillion	~76 years	Infeasible

As shown in Figure 2, brute force is O(n!), the fastest-growing complexity class. Any class exceeding roughly 12 students makes it computationally impractical. The approach is complete and correct, but only useful at a small scale (Cormen et al., 2022).

5. Task 3 - Heuristic (Smart) Approach

5.1 Two Practical Strategies

Heuristic 1: Most-Constrained Student First (MRV)

Rank students by total constraint count (friend pairs + city-mates). Place the most-constrained student first. For each subsequent seat, assign the first candidate satisfying both constraints with the placed neighbour. Backtrack one step if no valid candidate exists.

Heuristic 2: City-Separation Interleaving

Group students by city, then interleave groups across seats (City A, City B, City C, City A, ...). This eliminates city-conflict violations by design, reducing the problem to friend-constraint resolution only.

Figure 3 illustrates both strategies step by step, compares them against brute force, and shows an example valid seating output.

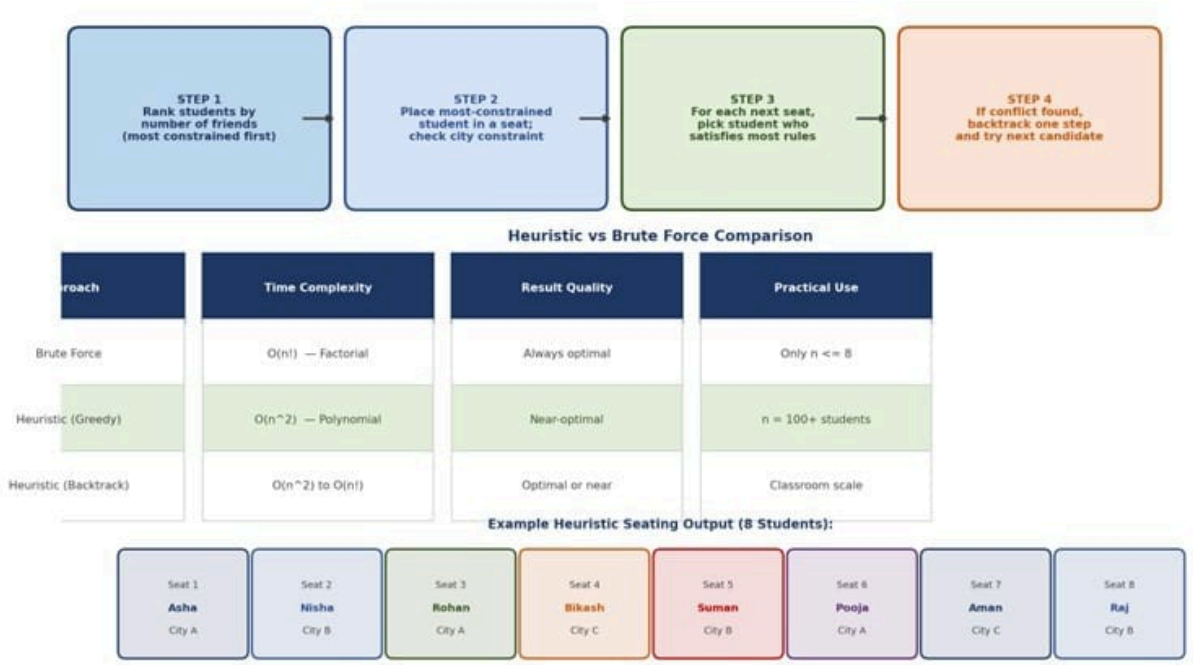

Figure 3: Heuristic Strategies Step-by-Step Flow, Comparison Table, and Example Seating Output

5.2 Time Reduction and Trade-offs

Heuristics reduce time complexity from O(n!) to approximately O(n²). The key mechanism is pruning: when a candidate violates a constraint at seat i, all (n − i − 1)! downstream arrangements are skipped immediately (Levitin, 2011). The trade-off is that heuristics are not complete a greedy early placement may make a later student impossible to seat. This is acceptable in practice: the teacher's goal is to reduce talking opportunities, not achieve mathematical perfection. Randomised restarts improve solution quality further at negligible cost.

6. Conclusion

The classroom seating arrangement problem clearly illustrates the P vs NP distinction. Verification is O(n) (Class P); finding a valid arrangement confronts an O(n!) search space with no known polynomial-time solution (Class NP), as shown in Figure 1. Brute force, illustrated in Figure 2, is complete but collapses beyond roughly 12 students. The heuristic strategies in Figure 3 deliver near-optimal results in O(n²), making any class size manageable. For teachers, heuristics provide practical seating arrangements even when mathematical perfection cannot be guaranteed; an imperfect arrangement still prevents the vast majority of conversations. The broader lesson: for NP problems, good-enough solutions found efficiently are far more valuable than perfect solutions that are computationally out of reach.

TASK 3: DATABASE NORMALISATION, ER MODELLING & SQL

1. Introduction

Effective data management is a cornerstone of reliable software systems. When data is stored without structure, inconsistencies emerge: the same information appears in multiple places, a single update requires changes across many rows, and deleting one record accidentally destroys unrelated information. Database normalisation is the systematic process that eliminates these problems by organising data into well-structured, logically independent tables.

This report analyses a flat, unnormalised college club membership table, identifies its structural problems, applies the three normal forms (1NF, 2NF, 3NF) to produce a clean relational schema, constructs an Entity Relationship (ER) diagram, and writes the SQL queries required to operate the normalised database.

2. Task 1 - Identifying Data Problems

Figure 1 annotates the original unnormalised table with its key data problems.

StudentID	StudentName	Email	ClubName	ClubRoom	ClubMentor	JoinDate
1	Asha	asha@email.com	Music Club	R101	Mr. Raman	1/10/2024
2	Bikash	bikash@email.com	Sports Club	R202	Ms. Sita	1/12/2024
1	Asha	asha@email.com	Sports Club	R202	Ms. Sita	1/15/2024
3	Nisha	nisha@email.com	Music Club	R101	Mr. Raman	1/20/2024
4	Rohan	rohan@email.com	Drama Club	R303	Mr. Kiran	1/18/2024
5	Suman	suman@email.com	Music Club	R101	Mr. Raman	1/22/2024
2	Bikash	bikash@email.com	Drama Club	R303	Mr. Kiran	1/25/2024
6	Pooja	pooja@email.com	Sports Club	R202	Ms. Sita	1/27/2024

Redundant Data: Asha (ID=1) repeats across rows 1 & 3

Update Anomaly: Music Club room/mentor copied in 3 rows

Redundant Data: Bikash (ID=2) repeats across rows 2 & 7

Insert Anomaly: Cannot record a new club until a student joins it — no student row means no club record.

Deletion Anomaly: Deleting Rohan (row 5, sole Drama Club member) permanently erases all Drama Club data.

Figure 1: Data Problems in the Original Unnormalised Club Membership Table

2.1 Three Problems in the Original Table

- Redundant data: Student information repeats in every row in which a student appears. Asha (StudentID 1) appears in rows 1 and 3 with an identical name and email; Bikash (StudentID 2) appears in rows 2 and 7.
- Mixed entity data: Student attributes and Club attributes are stored in the same table despite belonging to independent real-world entities.
- No proper primary key: No single column uniquely identifies each row. A composite key of (StudentID, ClubName) is the minimum required, but is not defined.

2.2 Anomaly Definitions

Insert Anomaly

A new club cannot be recorded until at least one student joins it. No student row means no club record.

Update Anomaly

Changing the Music Club's room requires updating three rows simultaneously. Missing even one leaves the database inconsistent.

Deletion Anomaly

Deleting Rohan, the only Drama Club member, permanently erases all Drama Club information from the database.

3. Task 2- Normalising the Data

Figure 2 illustrates the full normalisation journey from the unnormalised table through 1NF, 2NF, and 3NF.

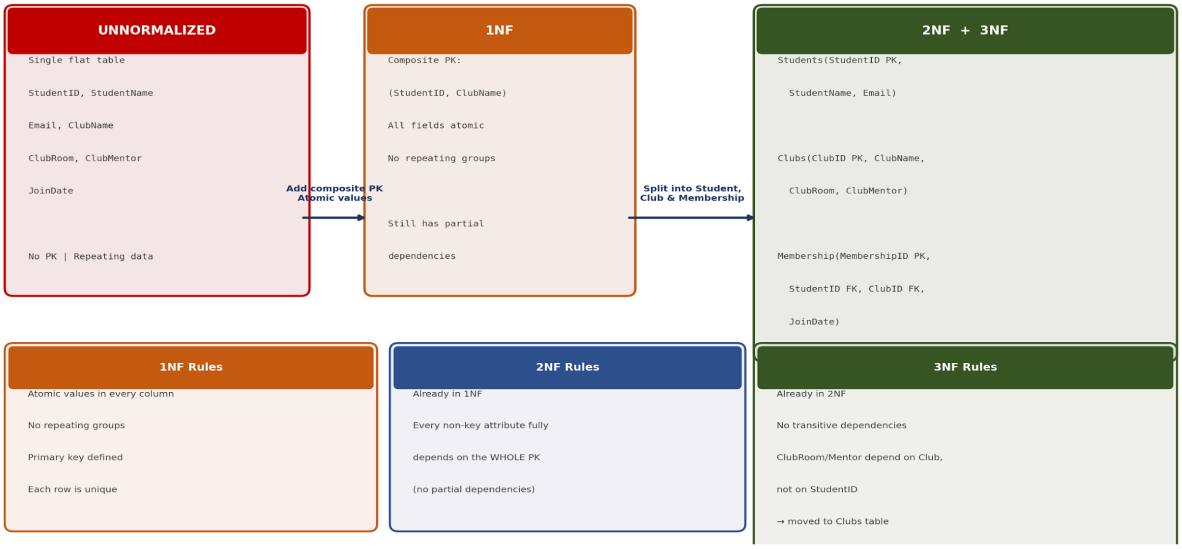

Figure 2: Normalisation Steps Unnormalised Table through 1NF, 2NF, and 3NF with Formal Rules

3.1 First Normal Form (1NF)

A table is in 1NF when every column holds atomic values, no repeating groups exist, and a primary key is defined (Codd, 1970). The original table has atomic values. To achieve 1NF, a composite primary key of (StudentID, ClubName) is declared. However, student details still partially depend only on StudentID and club details only on ClubName; partial dependencies are eliminated in 2NF.

3.2 Second Normal Form (2NF)

A table is in 2NF when in 1NF and every non-key attribute fully depends on the entire primary key (Date, 2003). Since the composite key is (StudentID, ClubName): StudentName and Email depend only on StudentID → moved to the Students table. ClubRoom and ClubMentor depend only on ClubName → moved to Clubs table. JoinDate depends on both → remains in the Membership table.

Table	Columns	Primary Key
Students	StudentID, StudentName, Email	StudentID
Clubs	ClubName, ClubRoom, ClubMentor	ClubName
Membership	StudentID, ClubName, JoinDate	(StudentID, ClubName)

3.3 Third Normal Form (3NF)

A table is in 3NF when in 2NF, with no transitive dependencies, no non-key attribute depends on another non-key attribute (Date, 2003). All 2NF tables satisfy this. Surrogate integer keys (ClubID, MembershipID) are introduced for efficient foreign key referencing. Final 3NF schema:

Table	Columns	Primary Key	Foreign Keys
Students	StudentID, StudentName, Email	StudentID	-
Clubs	ClubID, ClubName, ClubRoom, ClubMentor	ClubID	-
Membership	MembershipID, StudentID, ClubID, JoinDate	MembershipID	StudentID → Students; ClubID → Clubs

4. Task 3 - Entity Relationship (ER) Diagram

Figure 3 presents the ER diagram for the normalised system. Primary keys are underlined; foreign keys are marked (FK); crow-foot notation shows cardinality.

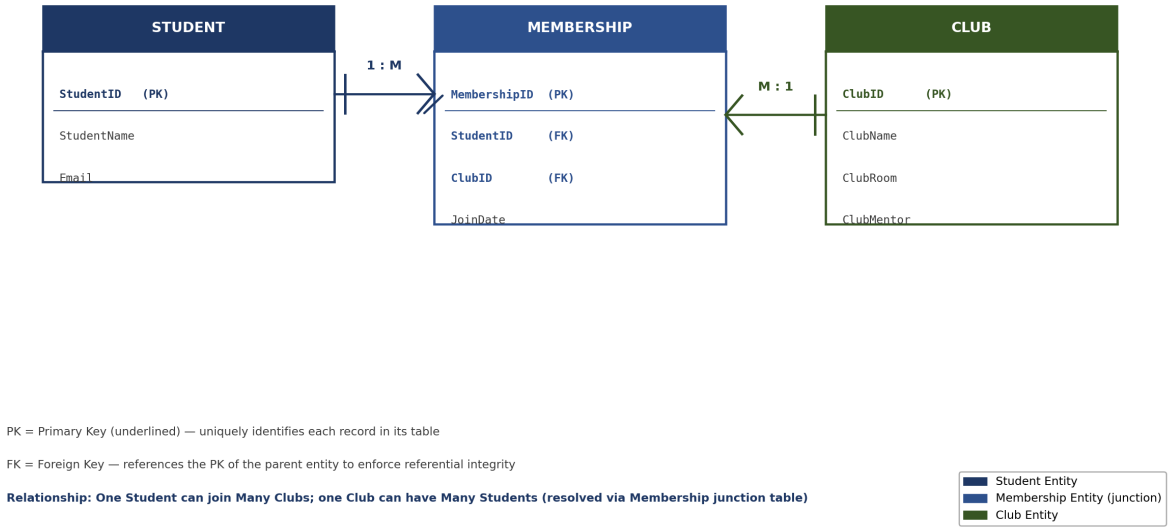

Figure 3: ER Diagram - College Club Membership System (Entities, Attributes, PK/FK, Cardinality)

As shown in Figure 3, one Student can join Many Clubs (1:M from Student to Membership); one Club can have Many Students (1:M from Club to Membership). The many-to-many relationship between Student and Club is resolved through the Membership junction table, the standard relational pattern for M:N relationships (Elmasri and Navathe, 2015).

5. Tasks 4 & 5 SQL Operations and JOIN

Figure 4 provides a complete visual reference for all SQL queries. All string values use consistent single-quote syntax.

Task 4a — INSERT New Student

```
INSERT INTO Students (StudentID, StudentName, Email)

VALUES (8, 'Priya', 'priya@email.com');
```

Task 4b — INSERT New Club

```
INSERT INTO Clubs (ClubID, ClubName, ClubRoom, ClubMentor)

VALUES (5, 'Art Club', 'R404', 'Ms. Lata');
```

Task 4c — SELECT All Students

```
SELECT StudentID, StudentName, Email

FROM   Students

ORDER BY StudentID;
```

Task 4d — SELECT All Clubs

```
SELECT ClubID, ClubName, ClubRoom, ClubMentor

FROM   Clubs

ORDER BY ClubName;
```

Task 5 — JOIN: Display Student Name, Club Name, and Join Date

```
SELECT
    s.StudentName,
    c.ClubName,
    m.JoinDate
FROM   Membership  m
INNER JOIN Students s  ON  m.StudentID = s.StudentID
INNER JOIN Clubs    c  ON  m.ClubID    = c.ClubID
ORDER BY s.StudentName, m.JoinDate;
```

JOIN logic: Membership (m) is the junction table linked to Students via StudentID and to Clubs via ClubID

Figure 4: SQL INSERT, SELECT, and JOIN Queries for the Normalised Club Membership Database

5.1 Basic SQL Operations (Task 4)

INSERT New Student

```
INSERT INTO Students (StudentID, StudentName, Email)
VALUES (8, 'Priya', 'priya@email.com');
```

INSERT New Club

```
INSERT INTO Clubs (ClubID, ClubName, ClubRoom, ClubMentor)
VALUES (5, 'Art Club', 'R404', 'Ms. Lata');
```

SELECT All Students

```
SELECT StudentID, StudentName, Email
FROM   Students
ORDER BY StudentID;
```

SELECT All Clubs

```
SELECT ClubID, ClubName, ClubRoom, ClubMentor
FROM   Clubs
ORDER BY ClubName;
```

5.2 JOIN Query (Task 5)

After normalisation, a JOIN query recombines related data at query time using the foreign key relationships (Silberschatz, Korth and Sudarshan, 2019). The Membership table bridges Students and Clubs via its two foreign keys:

JOIN — Student Name + Club Name + Join Date

```
SELECT
    s.StudentName,
    c.ClubName,
    m.JoinDate
FROM   Membership  m
INNER JOIN Students s  ON  m.StudentID = s.StudentID
INNER JOIN Clubs    c  ON  m.ClubID    = c.ClubID
ORDER BY s.StudentName, m.JoinDate;
```

The INNER JOIN returns only rows where matching records exist in all three tables. ORDER BY sorts results alphabetically by student name and then chronologically by join date within each student.

6. Task 6 - Reflection: Normalisation and JOINs

Normalisation eliminates redundancy by ensuring each fact is stored in exactly one place, a single source of truth. Update anomalies become impossible; insert and deletion anomalies

disappear (Connolly and Begg, 2014). JOINs are a direct and necessary consequence of normalisation: the cost of a JOIN executing in milliseconds is far smaller than the cost of managing redundant data that causes permanent data corruption. The normalised schema with JOINs is always preferable to a flat table with redundancy (Elmasri and Navathe, 2015).

7. Conclusion

The college club membership system demonstrates how normalisation transforms a flat, anomaly-prone table into a clean, reliable relational schema. The 1NF → 2NF → 3NF progression eliminates partial and transitive dependencies, the ER diagram in Figure 3 clarifies entity boundaries and relationships, and the SQL operations in Figure 4 confirm that the normalised schema is fully queryable. JOINs restore the relational connections that normalisation deliberately separates, making the schema both structurally sound and practically operational.

REFERENCES

Task 1 - Encoding Formats

[1] Josefsson, S. (2006) The Base16, Base32, and Base64 Data Encodings. RFC 4648. IETF. Available at: https://www.rfc-editor.org/rfc/rfc4648

[2] Freed, N. and Borenstein, N. (1996) MIME Part One: Format of Internet Message Bodies. RFC 2045. IETF. Available at: https://www.rfc-editor.org/rfc/rfc2045

[3] OWASP Foundation (2023) Double Encoding. OWASP Web Security Testing Guide. Available at: https://owasp.org/www-community/Double_Encoding

[4] HackerOne (2024) Mastering URL Encoding for Enhanced Web Security. Available at: https://www.hackerone.com/blog/secure-coding-practices-mastering-url-encoding-enhanced-web-security

[5] PortSwigger (2024) Obfuscating Attacks Using Encodings. Web Security Academy. Available at: https://portswigger.net/web-security/essential-skills/obfuscating-attacks-using-encodings

Task 2 - P vs NP

[6] Sipser, M. (2012) Introduction to the Theory of Computation. 3rd edn. Cengage Learning. ISBN: 978-1-133-18779-0.

[7] Cormen, T.H. et al. (2022). Introduction to Algorithms. 4th edn. MIT Press. ISBN: 978-0-26-204630-5.

[8] Garey, M.R. and Johnson, D.S. (1979) Computers and Intractability: A Guide to the Theory of NP-Completeness. W.H. Freeman. ISBN: 978-0-71-670024-4.

[9] Levitin, A. (2011). Introduction to the Design and Analysis of Algorithms. 3rd edn. Pearson. ISBN: 978-0-13-231681-1.

Task 3 - Database Normalisation & SQL

[10] Connolly, T. and Begg, C. (2014) Database Systems: A Practical Approach to Design, Implementation, and Management. 6th edn. Pearson. ISBN: 978-1-292-06118-4.

[11] Codd, E.F. (1970) 'A Relational Model of Data for Large Shared Data Banks', Communications of the ACM, 13(6), pp. 377–387. Available at: https://dl.acm.org/doi/10.1145/362384.362685

[12] Date, C.J. (2003). An Introduction to Database Systems. 8th edn. Pearson Addison-Wesley. ISBN: 978-0-321-19784-9.

[13] Elmasri, R. and Navathe, S.B. (2015) Fundamentals of Database Systems. 7th edn. Pearson. ISBN: 978-0-133-97077-7.

[14] Silberschatz, A., Korth, H.F. and Sudarshan, S. (2019) Database System Concepts. 7th edn. McGraw-Hill. ISBN: 978-0-078-02215-9.